Multi-level Chess Game Design

Q.Q. Chen (2330034001), S.Q. Huang (2330034020), M.H. Li (2330034027)
Y.R. Liang (2330034030), Y.X. Tu (2330034048)
Beijing Normal University-Hong Kong Baptist University United International College

***Abstract*—This project develops an intelligent chess game that leverages advanced artificial intelligence (AI) techniques to improve decision-making and overall gameplay. It features a multi-level adjustable difficulty system with six distinct tiers—Freshman, Simple, Medium, Nightmare, Master, and Genius—designed to offer players a tailored experience regardless of skill level. The AI framework integrates a variety of state-of-the-art models, including the Minimax algorithm, basic Convolutional Neural Networks (CNN), Monte Carlo Tree Search (MCTS) with Stockfish, hybrid MCTS-CNN, and Q-learning. These models work collaboratively to simulate millions of potential moves, analyze game states, and predict optimal strategies, creating a challenging yet engaging opponent. Training is based on real-world chess data sourced from lichess.com, allowing the AI to continuously refine its play through competitive interactions. By having models compete against one another, the system learns to adjust difficulty levels and deliver a progressively challenging experience for players. This project not only provides an immersive human-to-machine (H2M) gameplay experience but also contributes to the field of AI by examining how artificial intelligence interacts with human decision-making in complex strategic environments.**

***Index Terms*—Minimax Algorithm, Convolutional Neural Network (CNN), Monte Carlo Tree Search (MCTS), Q-learning, Deep Q-Network (DQN), Uniform Manifold Approximation and Projection (UMAP).**

I. INTRODUCTION

Chess is an ancient and complex board game that originated in 6th-century India, initially known as "Chaturanga" [1]. In the 21st century, the rise of artificial intelligence (AI) has added a new chapter to the history of chess. Through cutting-edge technologies, AI has transcended the role of mere chess analysis tools, evolving into a competitor capable of defeating top human players. Landmark achievements, such as IBM's Deep Blue defeating World Champion Garry Kasparov in 1997 [2] and the subsequent advancements seen in programs like AlphaZero [3], have revolutionized the way chess is understood and played. AI's ability to simulate millions of potential moves and predict optimal strategies has not only reshaped competitive play but has also led to innovative training methods, offering players an unprecedented depth of analysis.

The objective of this project is to develop an intelligent chess game utilizing Python, HTML, and CSS, incorporating a multi-level adjustable difficulty system with six tiers: Freshman, Simple, Medium, Nightmare, Master, and Genius. The AI framework leverages advanced techniques such as Convolutional Neural Networks (CNN), Monte Carlo Tree Search (MCTS), and Reinforcement Learning (RL), both as standalone models and in hybrid configurations, to enhance decision-making accuracy and gameplay dynamics. Training is conducted using data from lichess.com, with models competing against each other to calibrate and define the six difficulty levels.

In addition to its core functionality, this project aims to develop a dynamic AI that adapts to different playstyles, providing an engaging experience for players of varying skill levels. The game supports both human-to-machine (H2M) and machine-to-machine (M2M) modes, with an adjustable difficulty system to facilitate gradual progression. By integrating advanced machine learning techniques, the project contributes to both chess entertainment and AI research, particularly in understanding AI's interaction with human decision-making in complex strategic environments.

II. RELATED WORKS AND EXISTING TECHNIQUES

Early chess AI research focused on designing search strategies with heuristics. Initially, engines explored game trees exhaustively, with alpha-beta pruning improving efficiency by eliminating non-promising branches for deeper searches. Over time, evaluation functions became more sophisticated, considering not just basic positional factors but also aspects like piece activity, king safety, and pawn structure.

Convolutional neural networks (CNNs) have been successful in AI tasks, including Go, where models like CNN-Go predict professional moves by analyzing subtle changes in board positions [5]. However, CNNs struggle with long-term strategy, dynamic piece interactions, and creative play, focusing more on local objectives such as piece captures.

Monte Carlo Tree Search (MCTS), introduced by Kocsis and Szepesvári (2006) and Coulom (2007), was a breakthrough for game-playing AI, significantly improving performance in Go and adaptable to chess with heuristics to speed up convergence [8, 9]. Reinforcement learning (RL) methods like Q-Learning have also been explored, though they still face challenges in large-scale domains like chess due to the vast state space and low learning efficiency [10].

III. METHODOLOGY

A. Data Preprocessing

To process PGN chess record files and convert the chess game states into a format suitable for input into a neural network, we need to break down the task into several steps. The main idea is to extract each move from the PGN file, transform the game state (the chessboard after each move) into a numerical format (such as a matrix or tensor), and prepare

it for the neural network. The general processing workflow is shown as follows (***Fig. 1***).

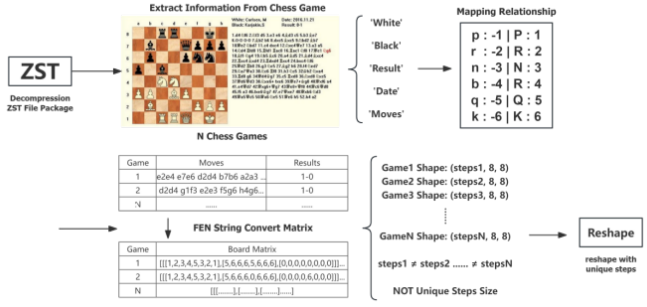


Fig. 1. Main Workflow of Chess Data Preprocessing

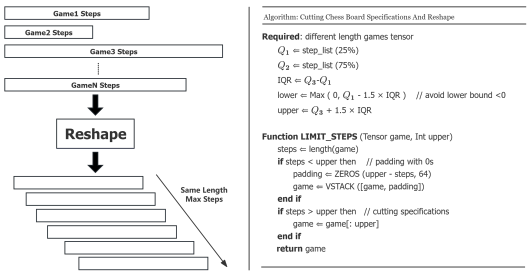


Fig. 2. Reshape Modules Processing Into Unique Step Size

After the above reshape process, it converts the reshaped board matrices into a 4D tensor format (*[steps, 8, 8, 12]*) suitable for input to a CNN where the last dimension represents 12 channels (one for each type of piece on the board). Due to the large size of the data, we try to batch the data and saves it in npy files to avoid memory overload issues for later use by the CNN model. The entire process is automated, using several stages to preprocess and reshape the chess data into a format that is compatible with neural network input. The data is first decompressed, extracted, and cleaned, followed by converting the chess game states (FEN) into matrices. These matrices are then reshaped to ensure consistent input sizes, and finally, the data is prepared and saved in batches for CNN training.

In MCTS-CNN model, we use five-dimensional data to input model. The use of five-dimensional data in this model is essential for handling chessboard states with temporal characteristics, where each time step represents the chessboard's state at a specific moment, and each state encapsulates the spatial features of the board, such as the number of channels, height, and width. This five-dimensional structure allows the model to process data from multiple time steps simultaneously, enabling it to capture the dynamic evolution of the chess game over time.

B. Minimax Algorithm

Before delving into the Minimax Algorithm, it is essential to first understand the theoretical framework underlying it. Game theory serves as the mathematical foundation for strategic interactions between participants, such as the black and white pieces in a two-player game. In this context, each player is assumed to pursue an optimal strategy, aiming to achieve the best possible outcome given the rules and available information governing the game (e.g., the movement rules and restrictions in chess).

In this setting, when it is the white player's turn, the goal is to maximize the player's score (i.e., maximize white's score). Conversely, during the black player's turn, the objective is to minimize the white player's score, thereby maximizing black's own score. This concept forms the basis of a two-party, zero-sum game, in which the players' strategies are driven by the desire to both maximize their own score and minimize their opponent's score. This dynamic leads to the following formula:

$$V(s_t) = \begin{cases} \text{evaluation}(s), & \text{if } s \text{ is a leaf node} \\ \dfrac{\max_{A \in V_{(alid)}} V(s_{t+1})}{a \in A(s_t)}, & \text{if turn is white} \\ \dfrac{\min_{B \in V_{(alid)}} V(s_{t+1})}{b \in B(s_t)}, & \text{if turn is black} \end{cases} \quad (1)$$

Represent state of each chess game as a tree node (s) that records the chess game information, and calculate the score evaluation value $V(s_t)$ for each chess game s_t:

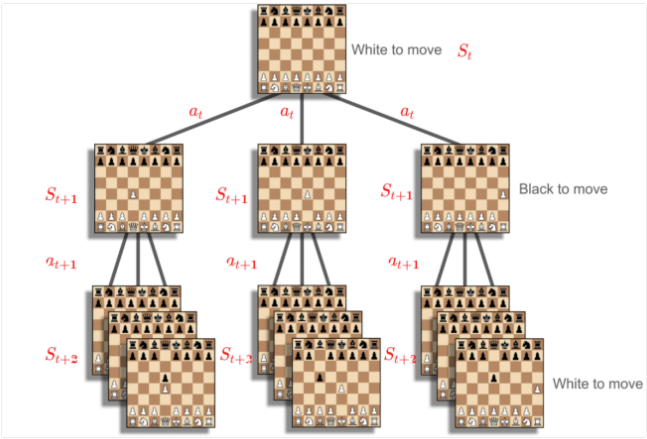


Fig. 3. The Chess Game State Tree And Scores With Respect To Certain Move

In adversarial games like Chess, Checker, Go, and Tic-tac-toe, a basic adversarial search algorithm called minimax is used to find the optimal action for the player while assuming the opponent also makes the optimal decision. This algorithm has been applied successfully in different two-player perfect-information zero-sum games [11].

C. CNN

A convolutional neural network (CNN) is a regularized type of feed-forward neural network that learns features by itself via filter (or kernel) optimization. This type of deep learning network has been applied to process and make predictions from many different types of data like image processing

[12]. Analyzing chess games similar as image data ***Fig. 4***is an innovative approach, especially in the context of modern deep learning and the convolutional neural networks (CNN). As mentioned in data preprocessing operation above, the state space of the chessboard at each time node has been converted into a (channels-first) array as the input of the model. According to research, Bitboard representation gives the best performance during the training of the data compared to regular ACN(Algebraic chess notations) format shown as fellows [13]:

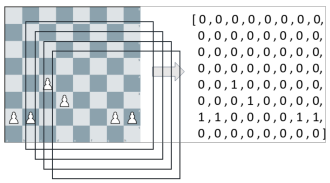


Fig. 4. The Bitboard Representation of White Pawn

According to some research, by leveraging the ability of CNNs to extract complex features from images, chess piece classification can be significantly improved. CNNs can capture local spatial patterns and hierarchical representations, allowing for the accurate identification of chess pieces with high precision and recall [14] . Drawing on the idea of CNNs to process image data input, this approach treats each position of the chessboard (and the information of the chess pieces) as a pixel or feature of the image, and uses methods similar with image processing techniques (like CNNs) to analyze the state of the chessboard and predict the next move. In a convolutional neural network (CNNs), the purpose of the convolution kernel on the board array is to extract local features from the situation ***(Fig. 5)***.

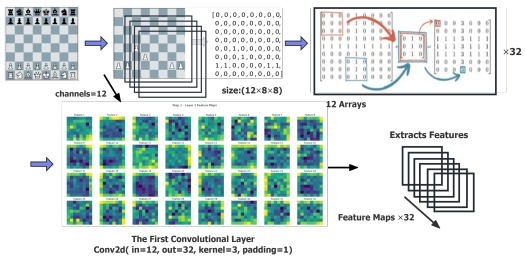


Fig. 5. CNNs Extracting Chess Game State's Features At Each Time Step

D. NNUE

The NNUE network takes two sparse binary arrays, each with 41,024 features, representing the "worldview" of the white and black kings. These arrays are encoded in a "player-relative coordinate system," ensuring that the evaluation process is independent of the side the engine plays, making the framework both flexible and efficient.

NNUE utilizes a shallow 4-layer neural network with integer parameters, optimized for fast matrix operations on Intel CPUs. Trained through supervised learning, it leverages target values from both alpha-beta search and self-play results to improve evaluation accuracy while minimizing computational overhead [15].

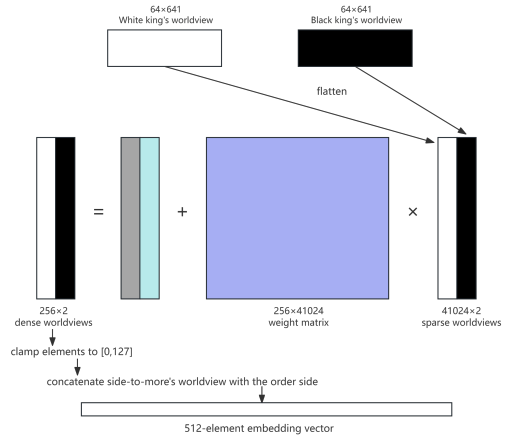


Fig. 6. The NNUE Input Layer

NNUE's efficiency is enhanced by two levels of sparsity: the sparse representation of non-king pieces in the input arrays and the fact that only a few entries change when a non-king piece is moved, reducing the need for expensive matrix-vector multiplications. This allows for efficient updates using simple vector additions or subtractions.

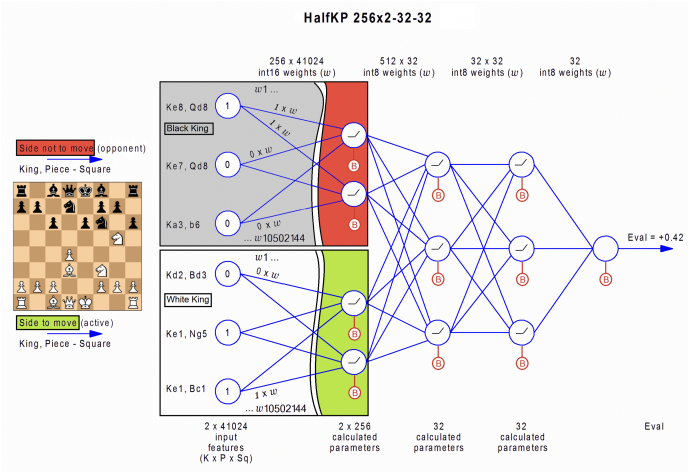


Fig. 7. The NNUE Architecture

E. Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is an effective technique for developing chess-playing algorithms or solving sequential decision-making problems in games [16]. The method employs a strategic tree search that balances exploration and exploitation, progressing through four essential phases: Selection, Expansion, Simulation, and Backpropagation.

Selection: The algorithm traverses the tree from root to leaf, balancing exploration and exploitation using the Upper Confidence Bound for Trees (UCT) formula to select nodes that maximize expected rewards and exploration.

$$UCT = \frac{w_i}{n_i} + C\sqrt{\frac{\ln N}{n_i}} \tag{2}$$

- w_i: The total reward (or wins) of the i-th child node.
- n_i: The number of times the i-th child node has been visited.
- N: The total number of simulations (or visits) at the parent node.
- C: A constant that controls the exploration vs. exploitation trade-off.

The goal of the UCT formula is to select the node that maximizes both the expected reward (based on previous simulations) and the potential for discovery (through exploration).

Expansion: The leaf node is expanded by generating child nodes corresponding to valid moves, adding new states for future exploration.

Simulation: A random rollout is run from the current node to a terminal state to estimate the outcome (win/loss/draw), providing a quality assessment of the node. This can be done using a simple random policy or a more advanced approach like a chess engine.

Update: The simulation result is propagated back through the tree, updating the win and visit counts for nodes along the path to the root, refining the decision-making for future iterations.

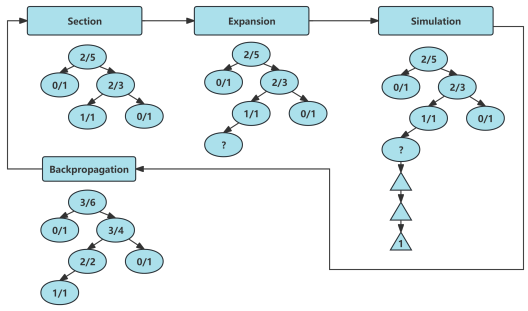


Fig. 8. The Four Steps of Monte Carlo Tree Search

Algorithm 1 Monte Carlo Tree Search

procedure MONTECARLOTREESEARCH(x_0)
  $r_0 \leftarrow$ MakeTree(x_0)
  repeat
    $r_i, r \leftarrow$ TreePolicy(r_0)
    $x_i \leftarrow r_i.state$
    $r \leftarrow$ DefaultPolicy(x_i)
    Backpropagate(r_i, r)
  until timeout
  return BestAction(r_0)
end procedure=0

procedure TREEPOLICY(r)
  $r \leftarrow 0$
  $x \leftarrow r.state$
  while Nonterminal(x) **do**
    $a \leftarrow$ SelectAction(x)
    $x' \leftarrow$ Transition$_p(x, a)$
    if $r.children[a][x'] =$ null **then**
      $r.children[a][x'] \leftarrow$ MakeTree(x')
      return $r.children[a][x'], r$
    end if
    $r \leftarrow r.children[a][x']$
    $x \leftarrow x'$
  end while
  return r, r
end procedure
procedure DEFAULTPOLICY(x, r)
  $r \leftarrow 0$
  while Nonterminal(x) **do**
    $a \leftarrow$ RandomAction(x)
    $x' \leftarrow$ Transition$_p(x, a)$
    $r \leftarrow r + R(x, a, x')$
  end while
  return r
end procedure
procedure BACKPROPAGATE(r, r)
  $r.reward \leftarrow r.reward + r$
  $r.count \leftarrow r.count + 1$
  $r \leftarrow r.parent$
  until $r =$ null
  return
end procedure
procedure BESTACTION(r)
  return $\arg\max_{a \in A} \frac{\sum r'.\text{children}[i][\cdot]^{r'.\text{reward}}}{\sum r'.\text{children}[i][\cdot]^{r'.\text{count}}}$
end procedure
=0

F. Q-Learning and AI agent

Chess game is a zero-game and the process of decision making. As for decision making: In some applications, AI agents can make decisions based on their inferences. Generally, the agent will base their decision on what is most likely to achieve a specified goal (like winning the best score of each moving step) [17]. An effective approach for AI agents to learn how to act in an environment is to go through trial-and-error experiences via interactions with the environment (like receiving certain score response after each single move step, maybe negative or positive).

Q Learning was first introduced by Watkins (1989) and is a model-free and off-policy RL algorithm. (**model-free:** model-free: Algorithm learns value function without an intermediate model; **off-policy:** Algorithm uses a greedy approach to the future move)

Q Learning guides decision-making by estimating the Q value of each state-action pair, and ultimately hopes to

learn the optimal action selection strategy by maximizing the cumulative return. The core of Q-learning lies in the Q value,$(Q(s_t, a_t))$ which is the value of a state-action pair, indicating the expected return (reward) that can be obtained after taking a certain action in a specific state. Let's suppose during the process of chess board game, at any time step t, the chess game agent is in state at certain time step denoted as a_t. Therefore, the action-value function is represented as $Q(s_t, a_t)$. This Q value is updated every time the agent interacts with the environment, or after each chess move and get certain reward r_t from the board evaluation function, using equation below [18]:

$$Q^{new}(s_t, a_t) = \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} * \overbrace{\Big(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} * \underbrace{\max_a Q(s_{t+1}, a)}_{\text{est. future value}} - \underbrace{Q(s_t, a_t)}_{\text{old value}} \Big)}^{\text{temporal difference}}$$
$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{\text{new value (temporal difference target)}}$$

Fig. 9. The Q-Value updated equation: update Q for each move by temporal difference.

Building upon the foundation of Q-learning and its application in chess games, we now transition to a more advanced reinforcement learning algorithm—Deep Q-Network (DQN). Deep Q-Network (DQN) is an important algorithm in reinforcement learning, which combines Q-learning and deep learning techniques, the difference is that DQN uses a target network to provide Q-values and that it trains the policy network using batches of data from a replay memory through back-propagation with an optimizer algorithm [19]

The core idea of the DQN algorithm is that when the state and action space are high-dimensional and continuous, deep learning is used to build both a value network (by using deep neural networks to approximate the Q-value function based on Q-learning) that solves reinforcement learning tasks and a loss function [20], which is the objective function in the reinforcement learning method that calculates labels and networks. The deviation of the output minimises the loss function [21].

G. Evaluation

In this study, we employ two complementary approaches to evaluate chess game strength: a Basic Quantitative Evaluation Method and UMAP-Based Dimension Reduction for Feature Extraction.

The quantitative method assigns fixed values to chess pieces based on their importance, adjusting the total score according to the game phase (opening, middle game, endgame) and board position. The phase weight reflects the changing roles of pieces, while the position weight emphasizes central squares.

To address the limitations of this approach, UMAP is used for dimensionality reduction, mapping high-dimensional game features into a lower-dimensional space. K-means clustering groups similar board states, and CNNs are trained to predict the cluster label for new positions, refining the evaluation with cluster-specific score distributions.

IV. IMPLEMENTATION

A. User Interface Design

The user interface of the chess game provides a seamless and intuitive experience, offering six adjustable difficulty levels: Freshman, Simple, Medium, Nightmare, Master, and Genius. Each level is designed to challenge players according to their skill, with gameplay dynamics enhanced through different models.

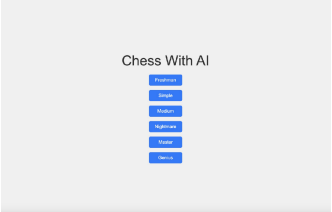


Fig. 10. Home Page For Selection



Fig. 11. Chess Game Interface

Once the difficulty level is selected, the user can begin the game. Upon hovering the mouse over a selected piece, the game will highlight the valid move locations for that piece, providing an intuitive interface for gameplay.

B. Minimax algorithm

In adversarial games like Chess, Checker, Go, and Tic-tac-toe, a basic adversarial search algorithm called minimax is used to find the optimal action for the player while assuming the opponent also makes the optimal decision. This algorithm has been applied successfully in different two-player perfect-information zero-sum games [22].

The minimax algorithm explores a game state tree to a predetermined depth, evaluating leaf nodes based on the game outcome (win: +1, draw: 0, loss: -1). Heuristic functions can also be used to assess non-terminal states. The Minimax Backtracking process starts at the leaf nodes and works backward to the root node, evaluating each layer based on the previous layer's values. The root node's value represents the best decision, and the best move is selected accordingly. The main workflow between white and black pieces follows this process.

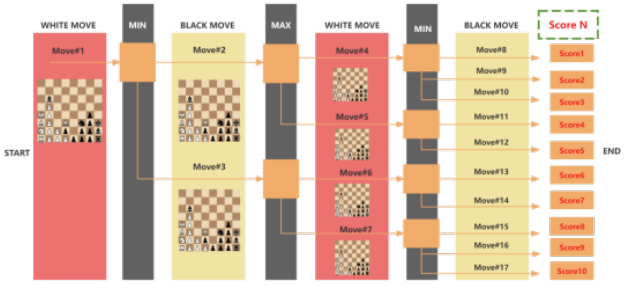


Fig. 12. The Main Workflow of Minimax Algorithm And Best Move's Selection

A major drawback of the Minimax algorithm is that it has high computational complexity because it must fully expand

the entire game tree. So the time complexity and storage space complexity required to represent and store multiple possible states of the chess game are very high just as fellows:

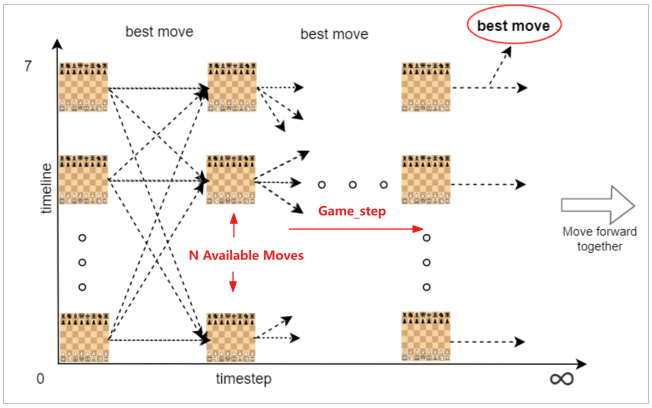


Fig. 13. Minimax Requires High Computational Complexity to Explore Moves

To address the high computational and storage complexity, Alpha-Beta Pruning was developed as an optimization method to reduce the number of nodes evaluated, enhancing algorithm efficiency. By eliminating branches that cannot influence the final decision, pruning skips unnecessary calculations without affecting the minimax outcome [24, 25].

Alpha-Beta pruning works by maintaining two values, Alpha and Beta, during the backtracking process of the Minimax algorithm, representing the current optimal estimates for the Max and Min players, respectively.

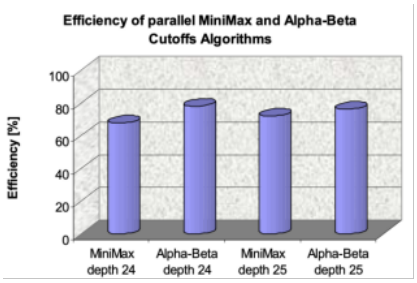


Fig. 14. Efficiency of implementations of minimax algorithm and Alpha-Beta Cutoffs

While Alpha-Beta pruning improves the efficiency of Minimax in many cases, its impact is limited in deep search scenarios, such as chess, where game complexity grows rapidly. In stages like the opening or endgame, despite pruning some branches, a large number of evaluations are still needed, reducing pruning's effectiveness (as shown in ***Fig. 14***). The benefits of pruning and parallel resource utilization increase with tree depth [26]. However, as the state space grows, Alpha-Beta pruning's advantages diminish, limiting overall performance.

To overcome these challenges, other strategies can be combined, such as dynamically adjusting the evaluation depth, heuristic methods, Monte Carlo Tree Search (MCTS), etc., to further improve the search efficiency.

Algorithm 2 Alpha-Beta Pruning Algorithm

```
Input: Node, Depth, IS_MAXIMIZE
if GAME OVER or Depth = 0 then
    return static evaluation of Node
end if
if IS_MAXIMIZE then {Max's turn}
    max_eval ← -
    for each child in Node do
        current_eval ← ALPHA_BETA(child, Depth-1, False)
        max_eval ← MAX(max_eval, current_eval)
        if max_eval ≥ beta then {Pruning condition}
            break {Prune the remaining children}
        end if
    end for
    return max_eval
else{Min's turn}
    min_eval ← +
    for each child in Node do
        current_eval ← ALPHA_BETA(child, Depth-1, True)
        min_eval ← MIN(min_eval, current_eval)
        if min_eval ≤ alpha then {Pruning condition}
            break {Prune the remaining children}
        end if
    end for
    return min_eval
end if

Initial call: ALPHA_BETA(Node current_state, Depth, True) =0
```

C. Basic CNN Model

The board state is represented as a $8 \times 8 \times 12$ tensor, where the first two dimensions, 8×8, correspond to the positions on the chessboard, and the third dimension, 12, represents the different types of chess pieces. Specifically, the 12 channels correspond to the following pieces: white pawns, black pawns, white knights, black knights, and so on for other piece types such as rooks, bishops, queens, and kings for both white and black players.

Through the convolution operation, the convolution kernel extracts features from local areas of the board. The main aspects that are extracted include:

1) **Local Chess Distribution:** This refers to the arrangement of pieces within a local region of the board, including formations such as lines of pawns (pawn chains), layouts of knights, and possible defenses and attacks formed by nearby pieces.
2) **Interactions Between Chess Pieces:** This includes capturing the interactions between pieces, such as two pieces supporting each other (e.g., a knight and a bishop protecting a square), or blocking each other's movement,

which can influence the strategic options available to players.

3) **Control of Important Locations:** This refers to areas of the board that are critical for strategy, such as the center of the board. The convolutional features can capture how well each player controls these areas, which is vital for determining the strength of their position.

These features help the neural network understand both the local and global relationships between pieces on the board and can be used to assess the current state and predict the best possible move.

The role of the convolution operation on the chessboard array is to extract local features of the chess game through the convolution kernel. These features can help the model recognize patterns on the board, the relative positions of the pieces, potential attack and defense relationships, and other important game information. Through multiple layers of convolution operations, CNN can gradually extract more abstract features, thereby improving the ability to evaluate the chess game and predict chess moves. The following is an introduction to the basic framework of the entire CNN model for chess games (***Fig. 15***).

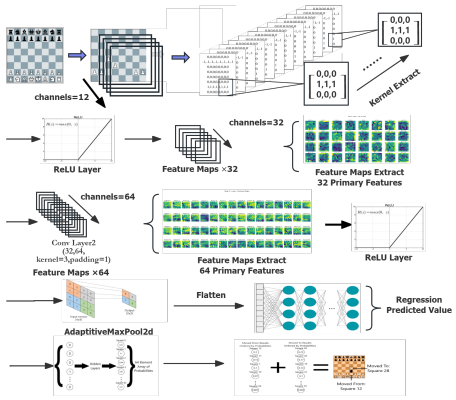


Fig. 15. CNN Based Chess Move Prediction Workflow

The entire CNN model process starts with input data (such as the state of the chessboard), extracts and integrates features through layers such as convolution, pooling, and full connection, and finally outputs prediction results (such as chess game evaluation or chess move prediction). During the training process, the model is continuously optimized through the back-propagation algorithm, and finally achieves better generalization ability and can make accurate predictions on new data.

D. MCTS with stockfish

In this section, we will critically examine the application of Stockfish in the evaluation of chess moves within the framework of Monte Carlo Tree Search (MCTS).

The NNUE evaluation function, employed by Stockfish, can be expressed as a multi-layer neural network applied to a feature vector. The evaluation of a given position is computed as:

$$E(x) = f\left(W_2 \cdot \text{ReLU}\left(W_1 \cdot x + b_1\right) + b_2\right) \tag{3}$$

Where:

- x is the input feature vector. In the context of chess, this might be the bitboard representation of the game state or another feature vector describing the board.
- W_1 and W_2 are the weight matrices for the first and second layers of the neural network, respectively.
- b_1 and b_2 are the bias terms for the first and second layers, respectively.
- `ReLU` is the Rectified Linear Unit activation function, defined as:

$$\text{ReLU}(z) = \max(0, z)$$

- f is the final activation function (or the output function).

Tapered scaling in Stockfish adjusts the weighting of evaluation criteria based on the game phase—opening, middle-game, or end-game. Each phase assigns different values to factors; for example, a passed pawn is more significant in the end-game than the middle-game. Stockfish uses two distinct weighting sets for the middle-game and end-game, with the transition controlled by a Phase factor ranging from 0 (game end) to 128 (game start), enabling smooth interpolation. Additionally, the evaluation is modified to account for the fifty-move rule, which triggers a draw if no captures or pawn moves occur within fifty moves. As this limit approaches, the evaluation gradually shifts toward a draw, reflecting the impending game conclusion.

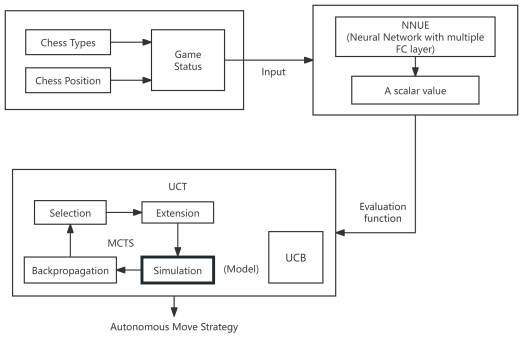


Fig. 16. MCTS (Stockfish) Model Workflow Diagram

In practice, the combination of Monte Carlo Tree Search (MCTS) and Stockfish operates as follows: During the selection and expansion phases, MCTS explores the game tree using the Upper Confidence Bounds for Trees (UCT) formula, which effectively balances exploration and exploitation. However, in the simulation phase, instead of relying on random moves, MCTS leverages Stockfish's evaluation function to assess the game state at each node and simulate the most probable outcomes.

In the backpropagation phase, the statistics of the nodes in the tree are updated based on Stockfish's evaluations of the board positions. This process is repeated for a predefined number of simulations or until a time limit is reached. The best move is then selected from the node with the highest average value, ensuring that the chosen move is not only likely to lead to a favorable outcome based on MCTS simulations but also strategically sound according to Stockfish's evaluation.

E. CNN-MCTS Model

In this study, we propose a hybrid model that integrates Monte Carlo Tree Search (MCTS) with Convolutional Neural Networks (CNN) to improve decision-making in the game of chess.

The core architecture consists of a chess state evaluation model based on a CNN, which is responsible for estimating the value of each move in a given game state. The CNN is designed to process the board's state representation as a tensor, using multiple convolutional layers to extract high-level features of the chessboard. The network's output consists of a set of probabilities that correspond to the likely success of each legal move.

MCTS is employed to explore the game tree and simulate potential moves in the search for the optimal move. The algorithm builds a tree of possible future states, where each node represents a game state, and each edge corresponds to a move. The tree is expanded by selecting moves according to the Upper Confidence Bound for Trees (UCT) formula, which balances exploration and exploitation. The formula is as following

$$\text{UCT Value} = \frac{\text{wins}}{\text{visits} + \gamma} + C \cdot \sqrt{\frac{\ln(\text{parent visits} + 1)}{\text{child visits}}}$$

- **wins**: The number of successful simulations for the current move.
- **visits**: The total number of simulations for the current move.
- γ: A constant that smooths the exploitation term, preventing overemphasis on highly-visited moves.
- C: A constant controlling exploration. Larger C encourages more exploration, while smaller C focuses on exploitation.
- **parent visits**: The number of simulations for the parent node, influencing the exploration term.
- **child visits**: The number of simulations for the child node, favoring less-visited moves.

The proposed model integrates Monte Carlo Tree Search (MCTS) with Convolutional Neural Networks (CNNs) to enhance decision-making in chess. In this hybrid approach, a CNN is used at each node to generate a heuristic evaluation of the board state, guiding the search toward promising branches. The root node is initialized with the current game state, and the search process selects child nodes based on a combination of win rate, exploration factor, and the CNN-provided heuristic value. The CNN outputs a probability distribution over possible moves, which informs the heuristic function evaluating each child node.

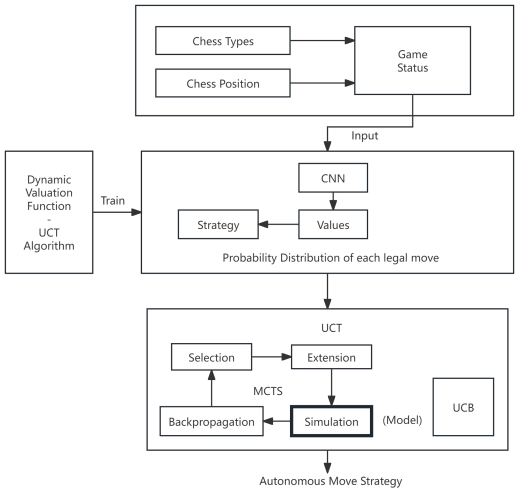


Fig. 17. MCTS-CNN Model Workflow Diagram

The UCT formula is applied to calculate a value for each child node, combining the win rate from previous simulations, an exploration term, and the heuristic score from the CNN. The search iterates through simulations, expanding the tree and updating node statistics (visits and wins). Once the search reaches the maximum iterations or depth, the best move is selected based on accumulated statistics and guided by the CNN policy.

This model combines the strategic exploration capability of MCTS with the precise evaluation of board positions offered by CNNs, enabling more efficient and accurate decision-making. The CNN allows MCTS to focus on the most promising branches, improving both the speed and quality of the search [27].

F. Basic Q-Learning

We have previously discussed the fundamentals of Q-learning. In this section, we will integrate it with AI Agent to build a model.

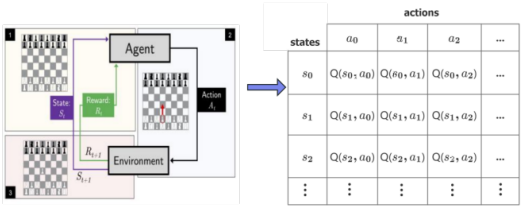


Fig. 18. The process of agent interactive with environment (left) by reward and stored procedures (right): *The state of the chess game is fed back to the agent through rewards and the agent will output the appropriate action (like the best move of certain pieces among all available moves) based on the reward feedback.*

At each step, the Q value of the current state-action pair is updated so that the optimal action can be selected in future decisions. Then, after computation, the state (s_t)and action (a_t) and the corresponding (s_t, a_t) pair of Q-value are stored in a table (initialized with all zeros). Through continuous iterative updates, Q-learning can eventually learn an approximate optimal strategy, that is, an action selection strategy that maximizes long-term rewards. In this way, Q Learning collects from the Q-table, we can discover the best possible state for each step. For a given state, the agent makes the best decision by maintaining certain balance between Exploration and Exploitation as fellows:

Algorithm 3 Epsilon Greedy Search Based Q-Learning

Require: white_agent, black_agent, initialized_board

```
while not GAME OVER do
    if board.move_turn = white then
        active_agent ← white_agent
    else if board.move_turn = black then
        active_agent ← black_agent
    end if
    move ← active_agent.FINDMOVE(
        epsilon: random choice from Valid() // Exploration
        1 - epsilon: arg // Exploitation
    )
    board.PUSH(move)
end while=0
```

Although Q-Learning is a fundamental RL algorithm, it faces limitations in chess due to the vast state space. A standard chessboard has 64 squares, each capable of holding different pieces, resulting in a state space with approximately 10^{40} possible configurations, even without considering piece color or special game rules. This immense complexity makes standard Q-learning impractical for chess.

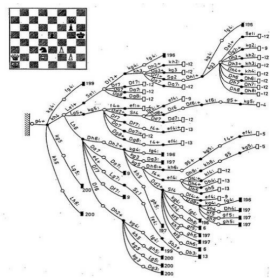


Table 1. Complexity of Go, Chess and Checkers games

Game	Board size	State space	Game tree size
Go	19 x 19	10^{172}	10^{360}
Chess	8 x 8	10^{50}	10^{123}
Checkers	8 x 8	10^{18}	10^{54}

Fig. 19. The Complex and Large Chessboard Movement States (left) And Complexity of Chess Computation Space (right): *Traditional Q-learning relies on a Q-table to store the Q value of each state-action pair. If the state space is very large, the size of the Q-table will become unacceptable, and the cost of storing and updating the Q-table will increase dramatically, making Q-learning ineffective.*

Therefore, ordinary Q-learning is usually not directly applicable to tasks such as chess. To overcome these problems, more advanced reinforcement learning algorithms such as Deep Q Network (DQN) can be used to combine deep learning and other techniques (such as experience replay and target network) to improve the learning efficiency and stability of the agent.

G. Deep Q-Network

The goal of Q-value function is to calculate the expected total reward for each state-action pair:

$$Q(s_t, a_t) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \cdot r_t \mid s_0 = s, a_0 = a\right] \tag{4}$$

where γ is the discount factor, and r_t is the time step t instant reward value.

The core idea of DQN is to use a deep neural network to approximate the Q-value function based on Q-learning.

$$Q(s_t, a_t) \approx Q_{NN}(s_t, a_t, \theta) \tag{5}$$

θ : the parameters of the Neural Network

After building up neural network approximated Q-value computation, then in each round of interaction with environment, agent chooses an action a_t based on current state s_t by using the epsilon-greedy strategy which is similar with the basic Q-learning method for keeping certain balance between Exploration and Exploitation to choose the best move.

$$\text{CHOOSE_MOVE} \begin{cases} \text{Prob1 } (\epsilon) & \Rightarrow \text{ random choice from Valid}(s_t) \\ \text{Prob2 } (1-\epsilon) & \Rightarrow \arg\max_{a \in \text{Valid}(s_t)} Q_{NN}(s_t, a_t) \end{cases} \tag{6}$$

After executing an action, the environment returns the next state s_{t+1} and the immediate reward $r_t$and stores this information in the replay pool. Finishing certain batches of rounds, randomly sample a mini-batch of experience quadruple from the replay pool to update the Q network based on computed experience quadruple (S_t , a_t, r_t , S_{t+1}) and loss function defined as fellow:

For each experience quadruple (S_t , a_t, r_t , S_{t+1}), compute its target Q value:

$$y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a', \theta^-) \tag{7}$$

Then, update parameters of Q network using the mean squared error loss (MSE):

$$L(\theta) = \frac{1}{N} \sum_{t=1}^{N} \left[y_t - Q_{NN}(s_t, a_t, \theta)\right]^2 \tag{8}$$

The calculation formula for network parameter update is as follows with workflow graph:

$$\theta \Leftarrow \theta - \alpha \cdot Grad_\theta \left(\frac{1}{N} \sum_{t=1}^{N} \left[y_t - Q_{NN}(s_t, a_t, \theta)\right]^2 \right) \tag{9}$$

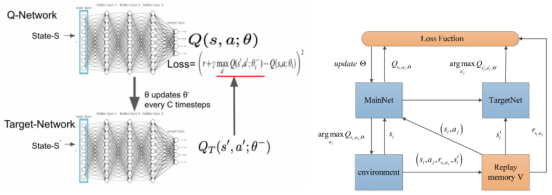


Fig. 20. Neural Network's parameters update process (left) and principle of DQN Algorithm (right): Adjust the parameters of the Q network through the back-propagation algorithm, then we can get update for neural network's parameters and successfully solved the problem of applying reinforcement learning in high-dimensional state space.

Adjust the parameters of the Q network through the back-propagation algorithm, then we can get update for neural network's parameters and successfully solved the problem of applying reinforcement learning in high-dimensional state space.

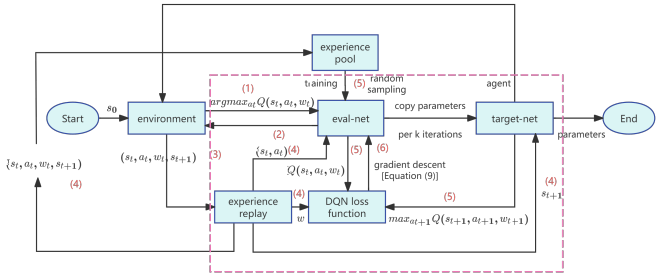


Fig. 21. Deep Q-Network Workflow Chart

```
----------------------------------------------------------------
      Layer (type)               Output Shape         Param #
================================================================
          Conv2d-1          [-1, 24, 8, 8]           2,616
            ReLU-2          [-1, 24, 8, 8]               0
          Conv2d-3          [-1, 32, 8, 8]           6,944
            ReLU-4          [-1, 32, 8, 8]               0
          Conv2d-5          [-1, 64, 8, 8]          18,496
            ReLU-6          [-1, 64, 8, 8]               0
          Conv2d-7         [-1, 128, 8, 8]          73,856
            ReLU-8         [-1, 128, 8, 8]               0
         Flatten-9              [-1, 8192]               0
         Linear-10                [-1, 64]         524,352
           ReLU-11                [-1, 64]               0
         Linear-12                [-1, 64]           4,160
================================================================
Total params: 630,424
Trainable params: 630,424
Non-trainable params: 0
```

Fig. 22. CNN-Based Network Parameters List

Algorithm 3 Training DQN for Chess Game

Initialized: Make Policy network weights θ randomly
Required: Clone policy network to create target network with weights θ' =0
for each episode **do**
  Initialize environment (Board) with random configuration of King, Rook position
  while not GAME OVER **do**
    if board.move_turn = white **then**
      Move white_agent.FINDMOVE()
      if ϵ: random choice from Valid()
      else: $1-\epsilon$
      Execute white_agent.move
      Execute optimal black_agent.move
      Observe reward R and next state S'
      Experience Tuple
      White Target Y
      White Update Policy Network by ADAM:
    end if
    if board.move_turn = black **then**
      Operation Symmetrical with the White Turn above
    end if
    Update Target Network Weights with new θ'
  end while
end for=0

H. Evaluation Part

1. Basic Quantitative Evaluation

In basic quantitative calculations, we analyze each part of the chess game in multiple dimensions:

A. Piece Values:

Different pieces have different values (as Table I) and they depends on the flexibility of chess pieces and the statistics of chess pieces' role in summarizing the experience of multiple chess games. So, to make more convenient computation, we decide to set constant value for each piece value:

TABLE I
CONSTANT VALUE FOR EACH PIECE AND SCORE

Chess Piece	Value	Chess Count	Total Score
chess.PAWN	1	8	8
chess.KNIGHT	3	2	6
chess.BISHOP	3	2	6
chess.ROOK	5	2	10
chess.QUEEN	9	1	9
chess.KING	∞	1	∞

$$\text{Score}_1 = \sum_{a \in \text{all_chess}} \text{Value}_a \times \text{Count}_a \tag{10}$$

B. Dynamically Adjust Value Scores:

Design a dynamic chess position scoring function that evaluates the strength of a position of a specific color (white or black) on the board. The goal is to dynamically adjust the score by multiple factors to reflect the complexity and actual strategic value of the position. Improve the accuracy of scoring by dynamically adjusting weights to distinguish between strong and weak situations.

a. Phase Weight:

Since the number of remaining pieces in the opening, middle game, and endgame is different in a chess game, and

these pieces play different roles in different chess games, we can estimate whether the current situation is the opening, middle game, or endgame by counting the number of high-value pieces (such as the queen and the rook) on the board, and use this to assign different value weights.

b. Position Weight:

Through the analysis of multiple chess games, we can conclude that the central core area of the chessboard is as follows (***Fig. 23***):

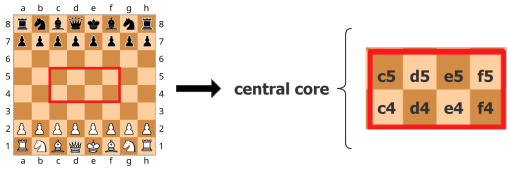


Fig. 23. Central Core Position of Board

c. Conservation and Threat Assessment

Protection of your own chess piece: If your own chess piece is protected by other chess pieces of your own side, you will get extra points. Threat from the enemy: If your own chess piece is attacked by the enemy, the score will be reduced.

d. King's Safety

By counting the enemy's control over the area on the board, the safety of the king can be indirectly quantified. The wider the enemy's attack range, the more the score will be reduced.

2.UMAP-Based Dimension Reduction For Feature Extraction

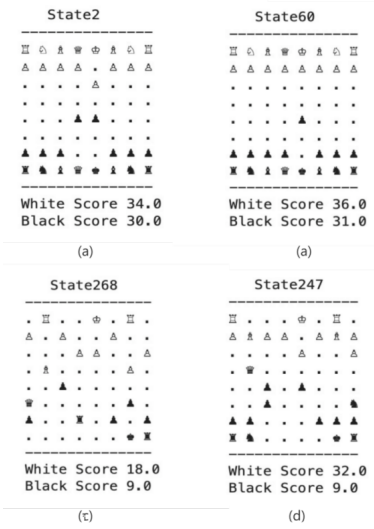


Fig. 24. Quantitative Evaluation Incorrectly Judgement Example

We observed that traditional chess evaluation algorithms can sometimes provide inaccurate assessments, particularly in positions like State 2 and State 60, where seemingly simple moves lead to significant score discrepancies. This issue arises from the subjectivity of the manually defined evaluation function, which is influenced by unstable and unpredictable factors.

For example, in State 268, Black has a more diverse set of moves due to the bishop and king, but the evaluation score fails to reflect this flexibility. Similarly, in State 247, Black controls the center and has a better chance of winning, yet the evaluation doesn't recognize this advantage.

To address these limitations, we propose transforming the evaluation into a clustering task. This involves grouping similar positions based on shared characteristics, offering a more intuitive understanding of position strength. We apply UMAP for dimensionality reduction, which preserves the topological structure of high-dimensional data while optimizing for a low-dimensional projection.

We evaluate clustering algorithms such as k-means, HDBSCAN, GMM, and Hierarchical Clustering on the low-dimensional feature space produced by UMAP. Our experiments with multiple image datasets show a significant improvement in clustering performance, with accuracy gains of up to 60%.

For this analysis, we use approximately 2000 games, apply UMAP for dimensionality reduction, and use k-means for clustering:

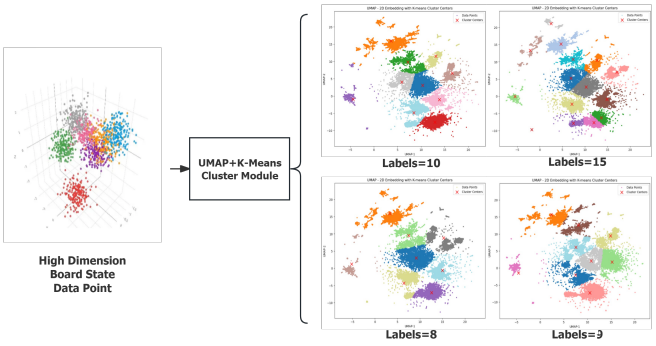


Fig. 25. UMAP-Based Dimension and K-Means Board State Cluster

After clustering the chess games, the chess games clustered together have similar characteristics, so calculating the score difference of this large group can more objectively reflect the differences between the two sides. At this time, the above quantitative evaluation method is used for calculation (Each Label represents one certain type of board state with certain characters):

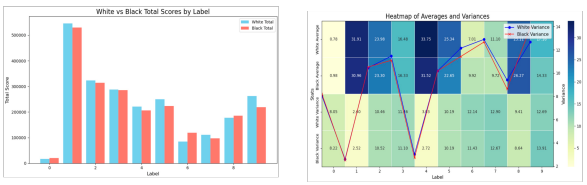


Fig. 26. Label Score Distribution and Evaluation Statics of Different Board State

In order to further refine the chess games corresponding to different Labels, we randomly selected 20,000 chess games for each Label, and further analyzed the differences in details of each game, so as to provide a more comprehensive evaluation for the players. So, we use evaluation function to evaluate score for each aspect.

State Label	Game Control	Central Region	King's Safety	Coordination
0 (White : Black)	W : B = 3.53 : 3.57	W : B = 2.31 : 4.22	W : B = 4.12 : 4.13	W : B = 2.41 : 4.12
1 (White : Black)	W : B = 3.53 : 3.53	W : B = 4.11 : 2.31	W : B = 4.22 : 4.22	W : B = 4.11 : 3.13
2 (White : Black)	W : B = 3.32 : 3.47	W : B = 4.12 : 3.17	W : B = 4.25 : 4.27	W : B = 4.01 : 4.13
3 (White : Black)	W : B = 2.62 : 4.21	W : B = 2.42 : 4.32	W : B = 3.21 : 4.11	W : B = 2.31 : 4.32
4 (White : Black)	W : B = 3.52 : 3.51	W : B = 4.11 : 3.12	W : B = 4.46 : 4.31	W : B = 3.12 : 4.14
5 (White : Black)	W : B = 4.22 : 2.42	W : B = 4.21 : 1.33	W : B = 5.03 : 3.23	W : B = 4.21 : 2.13
6 (White : Black)	W : B = 4.32 : 2.21	W : B = 5.01 : 1.13	W : B = 5.01 : 3.18	W : B = 5.02 : 2.32
7 (White : Black)	W : B = 4.13 : 2.31	W : B = 5.02 : 1.02	W : B = 5.02 : 2.33	W : B = 5.38 : 2.11
8 (White : Black)	W : B = 4.34 : 2.93	W : B = 5.15 : 1.33	W : B = 5.02 : 3.31	W : B = 5.02 : 2.12
9 (White : Black)	W : B = 4.41 : 2.11	W : B = 5.01 : 2.13	W : B = 5.01 : 3.23	W : B = 5.02 : 3.01

TABLE II
EACH LABEL OF DETAILED QUANTITATIVE EVALUATION OF THE BOARD STATE

Next, we have a label obtained by clustering algorithm. We can use CNN to learn the features of chess games with different labels, so that we can judge which label a new chess game belongs to. In this way, we can judge each chess game in a game, and count and integrate the number of times each game appears, so as to roughly judge the ability of both sides and the difference in the details of the chess game.

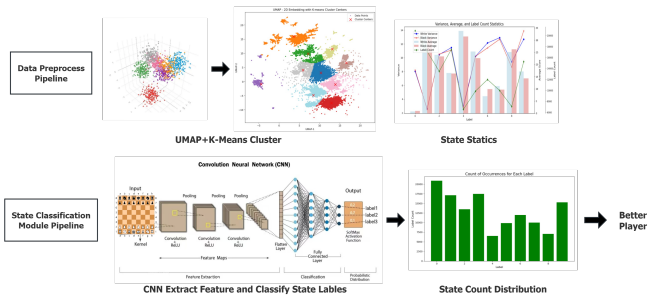


Fig. 27. Data Preprocessing Pipeline and State Classification Pipeline

Above two methods: quantitative evaluation function and UMAP-Based dimension reduction clustering module (***Fig. 27***), we can evaluate the advantages and disadvantages of each side after making their moves, and through multiple evaluation simulations, we can roughly distinguish the ability of the black and white sides to make their moves.

V. EXPERIMENTAL AND PERFORMANCE ANLAYSIS

We developed two chessboard evaluation methods to assess model performance and adjust difficulty levels for both AI and human players. These methods are outlined in the previous section. Model difficulty is ranked through self-play, allowing users to experience the game at different levels against various AI models.

We tested multiple model pairs, including Minimax vs CNN, Minimax vs MCTS, CNN vs CNN-MCTS, Q-Learning vs CNN, Q-Learning vs CNN-MCTS, DQN vs CNN-MCTS, and MCTS vs CNN.

The first evaluation method assigns weights to factors such as game stage, piece positioning, protection, threat assessment, and king security to form a comprehensive reward function. This function evaluates states and actions, guiding the model in strategy optimization. The figure below shows the evolution of the reward function in the Q-Learning model during gameplay.

The reward function plays a key role in the final score evaluation by observing the performance of both players and flexibly adjusting its corresponding value.



Fig. 28. Reward History Over Time

Below is the score display of some model game results:

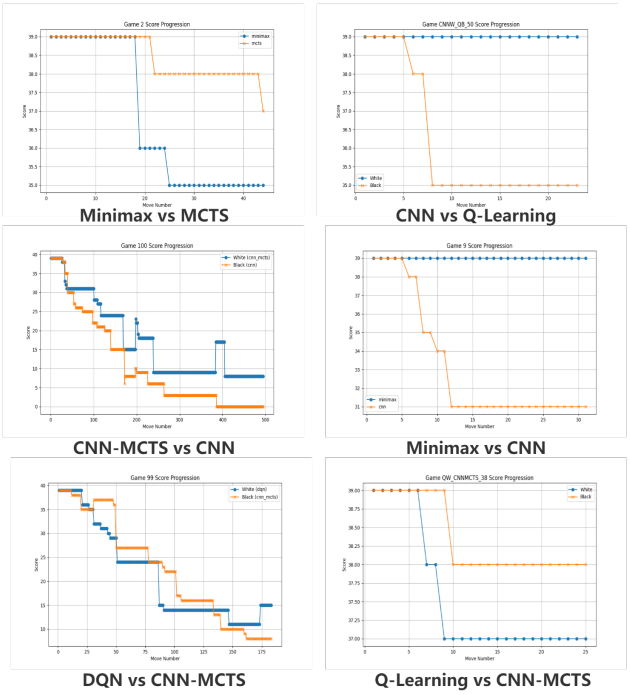


Fig. 29. Game Results

We initially set the model games to 100 rounds to rank the models based on their win rates. However, due to the large number of models and their dominance in different matchups, it was difficult to determine a clear ranking. To simplify, we used Minimax as the core model and compared it with five other models, resulting in the following win chart as following.

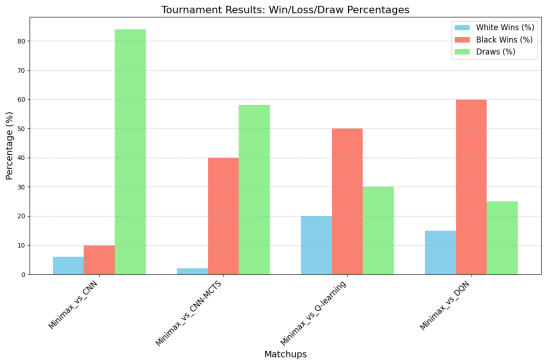


Fig. 30. Win rate ranking chart

The final ranking of models, from most to least difficult, is as follows: DQN, CNN-MCTS, MCTS, Minimax, CNN, Q-Learning.

Minimax is very suitable for complete information games such as board games, which can find optimal solutions. And the algorithm is simple, easy to understand and implement.

Traditional Q-Learning algorithms usually rely on discretization of state Spaces, which can lead to inefficiency and difficulty in dealing with complex chess games in high-dimensional or continuous state Spaces.CNN models usually have stronger feature extraction and generalization ability when dealing with board games, which makes them often have an advantage against Q-Learning based models. However, this is not to say that Q-Learning is not useful, in fact, Q-Learning variants that incorporate deep learning techniques (such as DQN, etc.) still perform well in many tasks.

CNN-MCTS and MCTS perform well in complex chess games, especially CNN-MCTS combines the feature extraction ability of deep learning and the search ability of MCTS. With the support of both, CNN-MCTS model shows strong chess ability.

VI. FUTURE WORK

In the future, there are several avenues for improving and enhancing the chess game design, which will contribute to its robustness, user engagement, and overall functionality. These include the following key areas:

A. Enhancing the Existing Models

A key future direction is integrating time series models into AI training. By using time series analysis, the AI can better predict game progression, anticipate future moves, and develop deeper strategies based on historical data. This approach allows the AI to learn from both the current board state and patterns in previous moves, enhancing its decision-making abilities.

B. User Interface Enhancements

Another improvement is enhancing the graphical user interface (GUI) to create a more user-friendly experience. Features like game history, move suggestions, and interactive tutorials would improve learning, especially for beginners. These additions would guide players, offer strategies, and track progress, making the game more engaging and accessible.

C. Multiplayer Functionality

To create a more interactive and social experience, adding multiplayer functionality is crucial. Online play would allow real-time challenges, with features like matchmaking, leaderboards, and chat systems fostering communication and community. These improvements would expand the game's appeal and make it a dynamic, social experience.

VII. CONCLUSION

In this study, we present an integrated framework for enhancing chess game decision-making using advanced machine learning techniques. By leveraging a large dataset, we differentiate between various levels of chess playing strength with models such as Minimax, Alpha-Beta Pruning, Convolutional Neural Networks (CNNs), Monte Carlo Tree Search (MCTS), Q-learning, and DQN. Our goal is to improve the performance of chess-playing AI systems in decision-making and evaluation across different difficulty levels.

We explored AI principles in game-playing, conducted a literature review, and developed a functional chess bot. The results demonstrate that the bot performs competitively, providing a challenging experience for human players.

Future enhancements include refining AI models by incorporating time series analysis, improving the graphical user interface (GUI) with features like game history and tutorials, and developing multiplayer functionality for real-time play and community interaction. These improvements will make the AI Chess Bot more engaging and accessible to players of all levels.

In conclusion, the integration of advanced machine learning models has significantly improved the AI system's decision-making and evaluation capabilities. With ongoing improvements in model refinement, user interface, and multiplayer features, the AI Chess Bot will offer an even more sophisticated and interactive experience, fostering both learning and competition.

REFERENCES

[1] D. Shenk, *The Immortal Game: A History of Chess*, Knopf Doubleday, 2007, p. 99.

[2] F.-H. Hsu, Behind Deep Blue, Princeton, NJ: Princeton Univ. Press, 2002, pp. 186, 217.

[3] X. Feng, Z. Wan, M. Wen, Y. Wen, W. Zhang, and J. Wang, "Alphazero-like Tree-Search can Guide Large Language Model Decoding and Training," *ArXiv*, vol. abs/2309.17179, 2023.

[4] S. Vrzina, "Piece by piece: Building a strong chess engine," Bachelor's thesis, Vrije Universiteit Amsterdam, Netherlands, 2023.

[5] B. Oshri, "Predicting moves in chess using convolutional neural networks," 2015.

[6] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," Machine Learning: ECML, 2006, pp. 282-293, doi: 10.1007/11871842_29.

[7] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in Computers and Games, H.J. van den Herik, P. Ciancarini, and H.H.L.M. Donkers, Eds. Berlin, Heidelberg: Springer, 2007, vol. 4630, pp. 43-54, doi: 10.1007/978-3-540-75538-8_7.

[8] S. Gelly and D. Silver, "Monte-Carlo tree search and rapid action value estimation in computer Go," Artificial Intelligence, vol. 175, pp. 1856-1875, 2011, doi: 10.1016/j.artint.2011.03.007.

[9] M. Lanctot, M. Winands, T. Pepels, and N. Sturtevant, "Monte Carlo tree search with heuristic evaluations using implicit minimax backups," in IEEE Conference on Computational Intelligence and Games (CIG), 2014, pp. 1-8, doi: 10.1109/CIG.2014.6932903.

[10] A. Castellini, D. Bragantini, D. Rossignolo, F. Segala, and A. Farinelli, "Tabular model learning in Monte Carlo tree search," 2024.

[11] V. H. Nguyen, "Board game AI and minimax algorithm," 2022.

[12] V. Chole, N. R. Gote, S. Ujwane, P. Hedaoo, and A. Umare, "Design and development of game playing system in chess using machine learning," Int. Res. J. Eng. Technol., vol. 8, 2021.

[13] M. Sabatelli, F. Bidoia, V. Codreanu, and M. Wiering, "Learning to evaluate chess positions with deep neural networks and limited lookahead," in Proc. ICPRAM, 2018, pp. 276-283.

[14] S. Parmar, "Chess piece classification via convolutional neural networks," in 2023 Int. Conf. Self Sustainable Artif. Intell. Syst. (ICSSAS), 2023, pp. 36-42.

[15] D. K. Klein, "Neural networks for chess," arXiv, abs/2209.01506, 2022.

[16] M. Świechowski, K. Godlewski, B. Sawicki, et al., "Monte Carlo tree search: A review of recent modifications and applications," Artif. Intell. Rev., vol. 56, no. 2, pp. 2497–2562, 2023, doi: 10.1007/s10462-022-10228-y.

[17] Z. Durante, Q. Huang, N. Wake, R. Gong, J. S. Park, B. Sarkar, et al., "Agent AI: Surveying the horizons of multimodal interaction," arXiv preprint arXiv:2401.03568, 2024.

[18] A.-I. Boar, "Solving chess endgames using Q learning," Ph.D. dissertation, 2022.

[19] J. Ypma, "Chess endgame analysis: Reinforcement learning vs. search tree algorithms," 2024.

[20] F. Zhou, L. Jin, and J. Dong, "Review of convolutional neural network," Chin. J. Comput., vol. 40, no. 6, pp. 1229–1251, 2017.

[21] Y. Yang, J. L. Juntao, and P. Lingling, "Multi-robot path planning based on a deep reinforcement learning DQN algorithm," CAAI Trans. Intell. Technol., vol. 5, no. 3, pp. 177–183, 2020.

[22] V. H. Nguyen, "Board game AI and minimax algorithm," 2022.

[23] I. Wilkens and J. El Khoury, "Minimax vs. Monte Carlo: A comparative case study on the use of minimax with alpha-beta pruning versus Monte Carlo tree search as decision-making algorithms in an Android chess application," 2024.

[24] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.

[25] S. S. Shevtekar, M. Malpe, and M. Bhaila, "Analysis of game tree search algorithms using minimax algorithm and alpha-beta pruning," International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 8, pp. 328–333, Nov. 2022. doi: 10.32628/CSEIT1228644. [Online]. Available: https://ijsrcseit.com/CSEIT1228644

[26] Y. Yang, L. Juntao, and P. Lingling, "Multi-robot path planning based on a deep reinforcement learning DQN algorithm," CAAI Transactions on Intelligence Technology, vol. 5, no. 3, pp. 177–183, 2020.

[27] J. Czech, J. Blüml, K. Kersting, and H. Steingrimsson, "Representation matters for mastering chess: Improved feature representation in AlphaZero outperforms switching to transformers," arXiv:2304.14918v2 [cs.AI], Aug. 2024.

[28] M. Allaoui, M. L. Kherfi, and A. Cheriet, "Considerably improving clustering algorithms using UMAP dimensionality reduction technique: a comparative study," in *International Conference on Image and Signal Processing*, Cham, Switzerland, 2020, pp. 317-325.